

PatchSim: a modular simulation framework for patch-based metapopulation epidemiology

Adish Assain Illikkal¹, Shreya Mukherjee¹, and Sneha S¹

¹ AI and Robotics Technology Park (ARTPARK), Indian Institute of Science, Bengaluru, India

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [✉](#)

Submitted: 22 August 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

PatchSim is an open-source Python framework for simulating infectious disease dynamics across spatially structured populations. It implements a patch-based metapopulation approach in which a landscape is partitioned into discrete geographical units (patches) — such as subdistricts, districts, or ecological zones — connected by a weighted contact network. Within each patch, disease progression is described by user-defined compartmental models (e.g., SIR, SEIR, SIRS) specified through a YAML configuration file using a declarative arrow-map transition syntax. PatchSim provides adaptive ODE and explicit-Euler solvers, Sobol sensitivity analysis via SALib (Herman & Usher, 2017), bounded least-squares calibration, and command-line and Python interfaces. Its configuration-first scope covers compartment-transfer models over supplied patch and group interaction data; scientific assumptions remain the modeller's responsibility. PatchSim is available on PyPI (`pip install patchsim`), with source code and documentation linked from the project record (Illikkal et al., 2026).

Statement of need

Mathematical modelling of infectious diseases is essential for understanding transmission dynamics, evaluating interventions, and informing public health policy (Keeling & Rohani, 2008). Metapopulation models — which capture the heterogeneity of disease spread across connected spatial units — are particularly valuable for diseases where population movement shapes outbreak trajectories, including foot-and-mouth disease in livestock, dengue in urban landscapes, and respiratory infections across administrative regions (Balcan et al., 2009; Grenfell & Harwood, 1997).

Implementing such models commonly requires custom scientific code or a platform tied to a particular modelling abstraction. This adds software work to the scientific tasks of specifying transitions, preparing spatial inputs, and checking model assumptions.

PatchSim addresses this gap through a configuration-first design in which model structure — compartments, transitions, and parameters — is declared in a YAML file rather than implemented in code. The intended users are epidemiological modellers, researchers studying spatial disease dynamics, and public health analysts who need to rapidly prototype and compare scenarios across spatial configurations and disease systems. PatchSim was developed at ARTPARK, IISc, to support active modelling work on livestock disease dynamics in India. The current package is used locally for ongoing foot-and-mouth disease vaccination-scenario analysis across Karnataka districts, replacing an earlier project-specific implementation; the workflow models cattle and buffalo populations stratified by age group and species.

State of the field

Existing software spans several related abstractions: EpiFire (Hladish et al., 2012) and EpiModel (Jenness et al., 2018) address contact-network epidemiology; epipack (Maier, 2021) supports compartmental, stochastic, and network models; EMOD (Bershteyn et al., 2018) uses an individual-based architecture; and GLEaMviz (Van den Broeck et al., 2011) represents global mobility-driven metapopulations. PatchSim instead centres a configuration-defined compartment system over a user-supplied weighted patch matrix and optional categorical group interactions.

PatchSim is an independent implementation, but it shares both its name and its modelling lineage with an earlier package developed at the Network Systems Science and Advanced Computing division of the University of Virginia (Venkatramanan et al., 2020). That package implements a metapopulation SEIR model and has been applied to seasonal influenza vaccine allocation (Venkatramanan et al., 2019) and to county-scale influenza forecasting in the United States (Venkatramanan et al., 2021). The earlier implementation was also used for age- and district-stratified COVID-19 vaccine allocation in Karnataka, as documented in a medRxiv preprint (Adiga et al., 2021). The approach was subsequently taken up in collaborative modelling work at the Indian Institute of Science and applied to foot-and-mouth disease in Indian livestock populations, which is the setting from which the present work grew. Where the earlier package provides a fixed SEIR compartmental structure serving a specific forecasting application, the framework described here generalises that approach: compartmental structure is treated as user-supplied configuration, so SIR, SEIR, SIRS, or arbitrary user-defined compartment sets are expressed in YAML without modifying source code. The name is retained to acknowledge this lineage. The earlier implementation is distributed as NSSAC/PatchSim; this package is distributed as patchsim from dsih-artpark/patchsim.

PatchSim is a lightweight, configuration-first Python framework for geographic metapopulation modelling with user-defined compartmental transitions. A modeller can switch between supported compartment structures, solvers, spatial networks, and group interaction inputs without rewriting the runtime.

Software design

PatchSim's central design principle is that compartmental model structure is declared in YAML rather than implemented in code. Transitions are expressed as arrow-map expressions pairing source and target compartments with a rate formula:

```
compartments: [S, I, R]
Parameters: {beta: 0.2, gamma: 0.1}
Transitions:
  "S -> I": "beta"
  "I -> R": "gamma * I"
```

For focal patch i , the infectious pressure is $\lambda_i(t) = \sum_j W_{ij} I_j(t) / N_j(t)$; the infection flow in this example is $\beta S_i \lambda_i$. PatchSim does not normalize the supplied weights. ODE mode uses `scipy.integrate.odeint`, SciPy's interface to LSODA, while discrete mode uses deterministic explicit Euler. Both evaluate the same derivative function; discrete results should be checked at successively smaller time steps. The current network is fixed from day zero. Patch populations, initial states, network weights, and optional group interactions are read from CSV files resolved relative to the configuration.

Validation checks required fields, identifiers, finite input values, population totals, matrix dimensions, transition endpoints, and an arithmetic-only expression language; these checks do not establish scientific validity. The JSON Schema is available for editors and external tooling. Reproducible analysis workflows provide seeded first- and total-order Sobol indices and

81 bounded multi-start calibration with input hashes and diagnostics. Built-in SIR, SEIR, SIRS,
82 and SIS templates and complete worked examples are included in the documentation. This
83 bounded modularity is configuration- and API-based; adding a numerical solver still requires
84 Python development. PatchSim is released under GPL-3.0.

85 AI usage disclosure

86 GitHub Copilot, OpenAI Codex CLI (GPT-5), and Anthropic Claude Code assisted with code,
87 tests, documentation, and copy-editing. CodeRabbit assisted with code review. Model versions
88 were not consistently retained for historical work. The authors reviewed, edited, tested, and
89 validated all assisted outputs and made the scientific and architectural decisions.

90 Acknowledgements

91 The authors thank Prof. Rajesh Sundaresan and Prof. Siva Athreya for their guidance on the
92 modelling work from which this framework grew, and Srinivasan Venkatramanan, whose earlier
93 metapopulation modelling work informed the approach taken here. The authors also thank the
94 ARTPARK team at the Indian Institute of Science for institutional support. This work was
95 supported by the AI and Robotics Technology Park, ARTPARK, IISc, Bengaluru. The authors
96 declare no conflicts of interest.

97 References

- 98 Adiga, A., Athreya, S., Lewis, B., Marathe, M. V., Rathod, N., Sundaresan, R., Swarup,
99 S., Venkatramanan, S., & Yasodharan, S. (2021). *Strategies to mitigate COVID-19*
100 *resurgence assuming immunity waning: A study for karnataka, india* [Preprint]. medRxiv.
101 <https://doi.org/10.1101/2021.05.26.21257836>
- 102 Balcan, D., Colizza, V., Gonçalves, B., Hu, H., Ramasco, J. J., & Vespignani, A. (2009).
103 Multiscale mobility networks and the spatial spreading of infectious diseases. *Proceedings*
104 *of the National Academy of Sciences*, 106(51), 21484–21489. [https://doi.org/10.1073/](https://doi.org/10.1073/pnas.0906910106)
105 [pnas.0906910106](https://doi.org/10.1073/pnas.0906910106)
- 106 Bershteyn, A., Gerardin, J., Bridenbecker, D., Lorton, C. W., Bloedow, J., Baker, R. S., &
107 others. (2018). Implementation and applications of EMOD, an individual-based multi-
108 disease modeling platform. *Pathogens and Disease*, 76(5), fty059. [https://doi.org/10.](https://doi.org/10.1093/femspd/fty059)
109 [1093/femspd/fty059](https://doi.org/10.1093/femspd/fty059)
- 110 Grenfell, B. T., & Harwood, J. (1997). (Meta)population dynamics of infectious diseases.
111 *Trends in Ecology & Evolution*, 12(10), 395–399. [https://doi.org/10.1016/S0169-5347\(97\)](https://doi.org/10.1016/S0169-5347(97)01174-9)
112 [01174-9](https://doi.org/10.1016/S0169-5347(97)01174-9)
- 113 Herman, J., & Usher, W. (2017). SALib: An open-source Python library for sensitivity analysis.
114 *Journal of Open Source Software*, 2(9), 97. <https://doi.org/10.21105/joss.00097>
- 115 Hladish, T., Melamud, E., Barrera, L. A., Galvani, A., & Meyers, L. A. (2012). EpiFire:
116 An open source C++ library and application for contact network epidemiology. *BMC*
117 *Bioinformatics*, 13, 76. <https://doi.org/10.1186/1471-2105-13-76>
- 118 Illikkal, A. A., Mukherjee, S., & S, S. (2026). *PatchSim: A modular simulation framework for*
119 *patch-based metapopulation epidemiology* (Version v0.1.0). Zenodo. [https://doi.org/10.](https://doi.org/10.5281/zenodo.22051463)
120 [5281/zenodo.22051463](https://doi.org/10.5281/zenodo.22051463)
- 121 Jenness, S. M., Goodreau, S. M., & Morris, M. (2018). EpiModel: An R package for
122 mathematical modeling of infectious disease over networks. *Journal of Statistical Software*,
123 84(8), 1–47. <https://doi.org/10.18637/jss.v084.i08>

- 124 Keeling, M. J., & Rohani, P. (2008). *Modeling infectious diseases in humans and animals*.
 125 Princeton University Press. ISBN: 9780691116174
- 126 Maier, B. F. (2021). Epipack: An open-source Python package for epidemiological
 127 compartmental models. *Journal of Open Source Software*, 6(60), 3097. <https://doi.org/10.21105/joss.03097>
 128
- 129 Van den Broeck, W., Gioannini, C., Gonçalves, B., Quagiotto, M., Colizza, V., & Vespignani,
 130 A. (2011). The GLEaMviz computational tool, a publicly available software to explore
 131 realistic epidemic spreading scenarios at the global scale. *BMC Infectious Diseases*, 11, 37.
 132 <https://doi.org/10.1186/1471-2334-11-37>
- 133 Venkatramanan, S., Chen, J., Fadikar, A., Gupta, S., Higdon, D., Lewis, B., Marathe, M.,
 134 Mortveit, H., & Vullikanti, A. (2019). Optimizing spatial allocation of seasonal influenza
 135 vaccine under temporal constraints. *PLOS Computational Biology*, 15(9), e1007111.
 136 <https://doi.org/10.1371/journal.pcbi.1007111>
- 137 Venkatramanan, S., Chen, J., & Marathe, M. (2020). *PatchSim: Code for simulating the*
 138 *metapopulation SEIR model* (Version v1.0). Zenodo. [https://doi.org/10.5281/zenodo.](https://doi.org/10.5281/zenodo.4313095)
 139 [4313095](https://doi.org/10.5281/zenodo.4313095)
- 140 Venkatramanan, S., Sadilek, A., Fadikar, A., Barrett, C. L., Biggerstaff, M., Chen, J.,
 141 Dotiwala, X., Eastham, P., Gipson, B., Higdon, D., Kucuktunc, O., Lieber, A., Lewis,
 142 B. L., Reynolds, Z., Vullikanti, A. K., Wang, L., & Marathe, M. (2021). Forecasting
 143 influenza activity using machine-learned mobility map. *Nature Communications*, 12(1),
 144 726. <https://doi.org/10.1038/s41467-021-21018-5>